

PROJECT DOCUMENTATION

POCKETSMART AI

Smart Budget & Recommendation Assistant

TEAM IDENTIFICATION

SWTID-2026-5457

TEAM SIZE

5 Members

PROJECT TEAM ROSTER

#	MEMBER NAME	PROJECT ROLE
1	Gayathri D	Team Leader
2	Sujithra K	Team Member
3	P Keerthi	Team Member
4	Priyadharshini K	Team Member
5	Aruna M	Team Member

Document Note: This document section identifies the project details and team composition for the *PocketSmart AI* project documentation.

POCKETSMART AI

Smart Budget & Recommendation Assistant

Project Documentation

Functional • Technical • User Workflow • Deployment Architecture

Document basis

This documentation is prepared from the supplied PocketSmart AI project document. The source is primarily visual and contains application screens, code excerpts, and an architecture diagram. The descriptions below preserve the terminology and capabilities visible in that source.

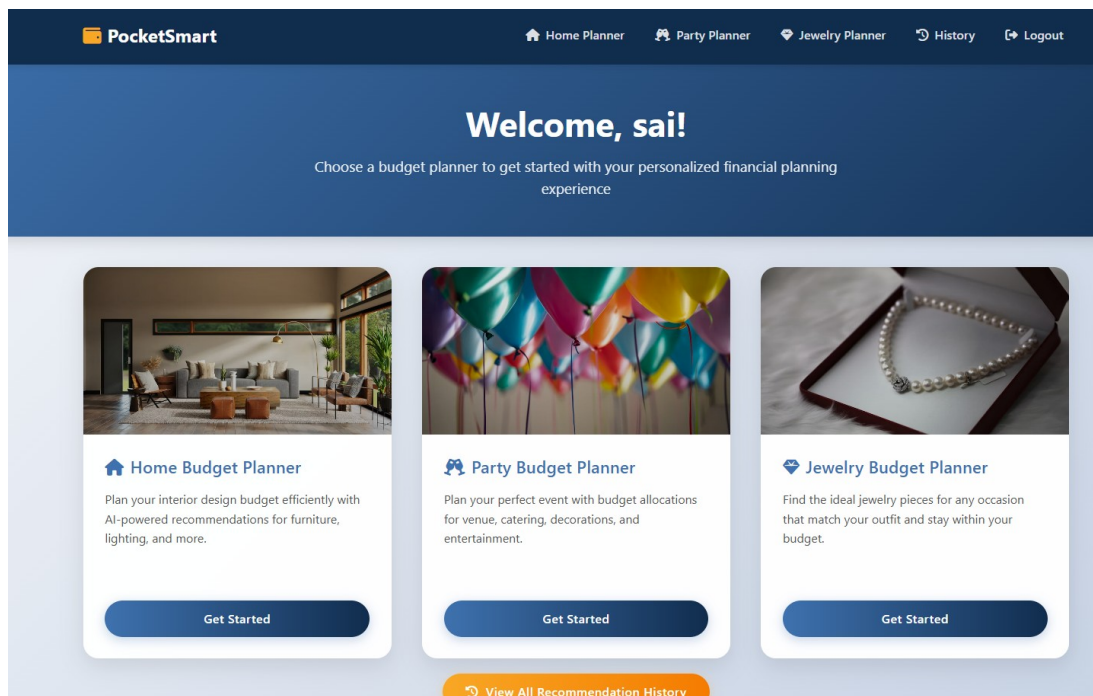


Figure 1. PocketSmart authenticated dashboard / planner selection screen.

Prepared from the supplied project reference

1. Document Control

Project	PocketSmart AI: Your Smart Budget & Recommendation Assistant
Document type	Software project documentation
Source	Supplied PocketSmart AI Word document
Primary scope	AI-assisted budget planning and personalized recommendations
Covered areas	Home interior, party planning, jewelry planning, recommendation history, authentication, and deployment architecture

2. Executive Summary

PocketSmart is presented as an AI-powered budget planning assistant designed to help users make personalized spending decisions across several everyday planning scenarios. The supplied application screens show three principal planners: Home Interior Budget Planner, Party Budget Planner, and Jewelry Budget Planner. Users provide a budget and scenario-specific preferences, then request personalized recommendations.

The system also provides recommendation history, authenticated access, product/shopping links, and additional suggestions intended to help users remain within their stated budget. The source architecture shows a web/mobile client fronted by Amazon CloudFront and Amazon Route 53, API traffic through Amazon API Gateway, backend orchestration through AWS Lambda, persistence in Amazon DynamoDB, user uploads/static assets in Amazon S3, credential management through AWS Secrets Manager, access control through AWS IAM, monitoring through Amazon CloudWatch, external commerce integrations, and Google Gemini as the generative recommendation engine.

3. Purpose and Objectives

- Provide a single interface for budget planning in multiple practical categories.
- Collect budget, quantity, preference, and scenario information from the user.
- Generate personalized budget allocations and product recommendations.
- Present remaining budget and category-level allocations clearly.
- Provide shopping links for recommended items where shown in the source.
- Allow users to review previous recommendations through recommendation history.
- Support authenticated access to the application.

4. Scope

The supplied source demonstrates the following in-scope functionality:

- User registration and sign-in screens.
- Authenticated dashboard and navigation.
- Home interior budget planning.
- Party/event budget planning.
- Jewelry budget planning with outfit-image upload.
- AI-generated budget plans and recommendations.
- Budget summary and remaining-budget display.
- Additional suggestions and styling tips.
- Recommendation history.
- Shopping links to multiple commerce platforms.
- Backend/API and cloud deployment architecture.

The source does not provide enough textual detail to define a complete formal API specification, database schema, test plan, or exhaustive non-functional requirements; those areas are therefore described only to the extent supported by the supplied screens and architecture.

5. Product Overview

5.1 Main User Journey

1. The user opens PocketSmart and signs in or creates an account.
2. The user reaches the dashboard and selects a planning category.
3. The user enters a budget and category-specific requirements.
4. The user submits the planner using the recommendation action.
5. The backend processes the request and uses the AI recommendation engine.
6. The application returns a personalized budget plan and recommendations.
7. The user reviews allocations, item details, prices, quantities, suggestions, and shopping links.
8. The recommendation can appear in recommendation history for later review.

5.2 Main Application Areas

Module	Purpose	Key Inputs	Key Outputs
Home Interior Planner	Plan an interior budget	Budget, lights/fixtures, fans, furniture, dining tables, rooms, preferences	Category allocations, items, prices, quantities, shopping links, suggestions
Party Planner	Plan an event budget	Budget, guests, party type, venue type, party needs, additional requirements	Budget plan and category recommendations
Jewelry Planner	Plan jewelry purchases	Budget, occasion, style preferences, outfit image	Outfit analysis, jewelry recommendations, styling tips
History	Review previous plans	Previously generated plans	Recent recommendation cards and plan summaries

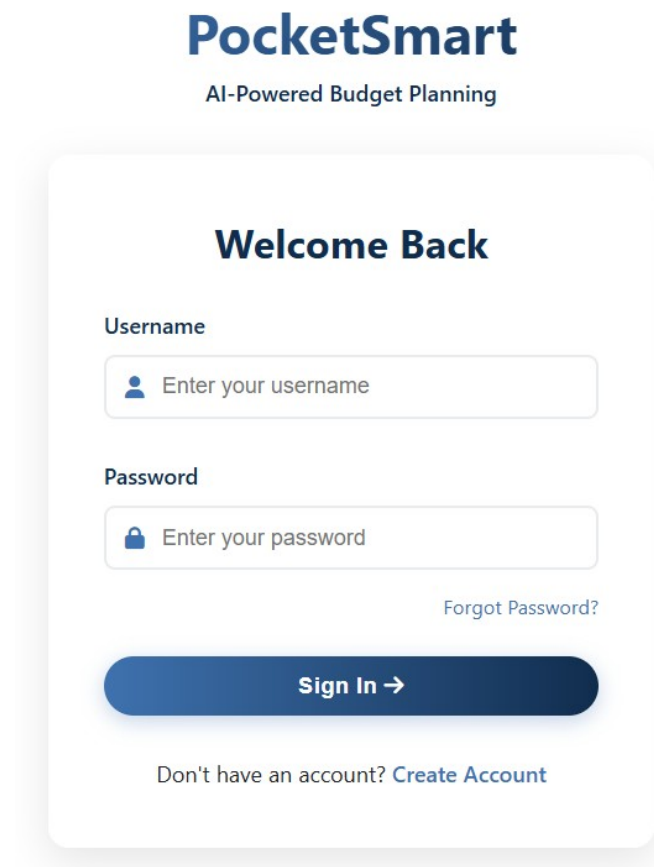
5.3 Navigation

The authenticated interface uses a top navigation bar. The supplied screens show links for Dashboard, Home Budget/Home Planner, Party Budget/Party Planner, Jewelry Budget/Jewelry Planner, History, and Logout.

6. Functional Module Documentation

6.1 Authentication and Account Access

The source includes a PocketSmart sign-in screen with username and password fields, a Forgot Password link, and a Create Account path. The registration screen collects username, email, password, and password confirmation.



The image shows a sign-in screen for 'PocketSmart', which is described as 'AI-Powered Budget Planning'. The screen has a white background with a subtle shadow. At the top, the 'PocketSmart' logo is in a bold, dark blue font, with the tagline 'AI-Powered Budget Planning' in a smaller, lighter blue font below it. The main heading 'Welcome Back' is centered in a bold, dark blue font. Below this, there are two input fields: 'Username' and 'Password'. The 'Username' field has a user icon and the placeholder text 'Enter your username'. The 'Password' field has a lock icon and the placeholder text 'Enter your password'. To the right of the password field is a link that says 'Forgot Password?'. Below the input fields is a large, dark blue button with the text 'Sign In →' in white. At the bottom, there is a link that says 'Don't have an account? [Create Account](#)'.

Figure 2. PocketSmart sign-in screen.

PocketSmart

AI-Powered Budget Planning

Create Your Account

Username

 Choose a username

Email

 Enter your email

Password

 Create a strong password

Confirm Password

 Confirm your password

Create Account 

Already have an account? [Sign In](#)

Figure 3. PocketSmart account registration screen.

A code excerpt in the source shows a FastAPI login route that checks for an existing token and redirects an already authenticated user to the dashboard. Another source excerpt shows JWT-related configuration using HS256 and an access-token expiration setting of 30 minutes.

```
# FastAPI app initialization
app = FastAPI(title="PocketSmart: AI Budget Planner")

SECRET_KEY = os.getenv("SECRET_KEY", "your_secret_key")
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30
```

Figure 4. FastAPI application initialization and JWT configuration shown in the source.

```

@app.get("/login", response_class=HTMLResponse)
async def login_page(request: Request):
    """Serve the login page"""
    # Check if user is already logged in
    try:
        token = await get_token(request)
        if token:
            user = await get_current_user(request, token)
            if user:
                return RedirectResponse(url="/dashboard", status_code=status.HTTP_302_FOUND)
    except Exception:
        pass

    return templates.TemplateResponse("login.html", {"request": request})

```

Figure 5. Login route and authenticated-user redirect logic shown in the source.

6.2 Dashboard

After authentication, the dashboard presents the three planning options and provides access to recommendation history. The screen describes PocketSmart as an AI-powered budget planning experience.

6.3 Home Interior Budget Planner

The Home Interior Budget Planner collects a total budget, counts for lights/fixtures, ceiling fans, furniture pieces, and dining tables, rooms to include, and special requirements or preferences.

PocketSmart

DashboardHome BudgetParty BudgetJewelry Budget

Logout

Home Interior Budget Planner

Create a customized budget plan for your dream home interior

Budget Details

Total Budget (\$)

5000

Fixtures & Furniture

Number of Lights/Fixtures

5

Number of Ceiling Fans

4

Number of Furniture Pieces

2

Number of Dining Tables

1

Rooms to Include

☒ Living Room

☒ Kitchen

☐ Bedroom

Additional Information

Special Requirements or Preferences

Any specific requirements or preferences...

Generate Recommendations

Figure 6. Home Interior Budget Planner input screen.

Example fields visible in the supplied screen:

- Total Budget
- Number of Lights/Fixtures
- Number of Ceiling Fans
- Number of Furniture Pieces
- Number of Dining Tables
- Rooms to Include: Living Room, Kitchen, Bedroom
- Special Requirements or Preferences
- Generate Recommendations

6.4 Home Interior Recommendation Output

The example output shows a Budget Summary with total and remaining budget, followed by category sections such as Lighting, Ceiling Fans, and Furniture. Each recommendation can include

an item name, description, price, quantity, and shopping links. An Additional Suggestions section provides budget-oriented advice.

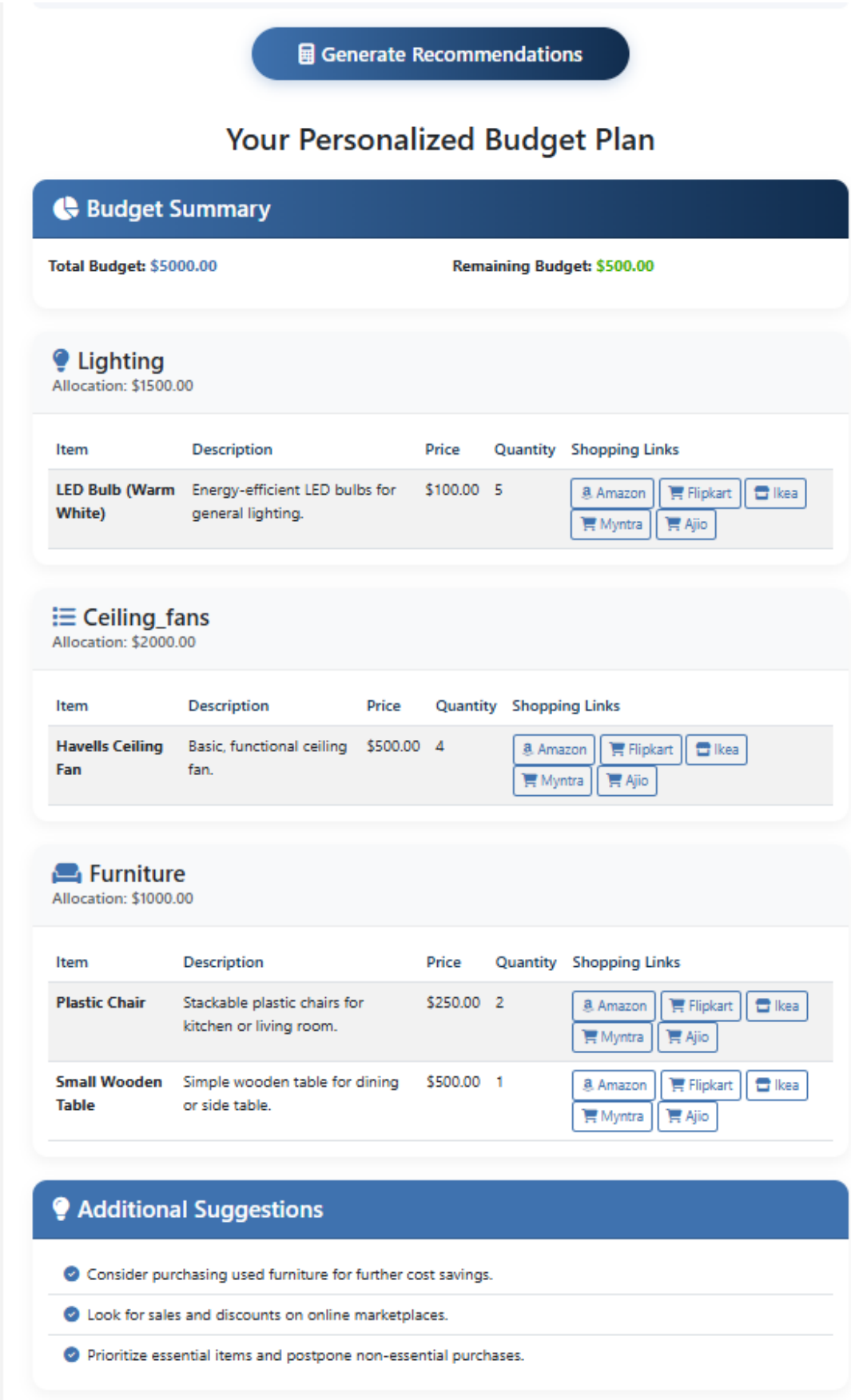


Figure 7. Example personalized home-interior budget plan.

6.5 Party Budget Planner

The Party Budget Planner captures basic budget and guest information, event details, party needs, and additional requirements. The supplied example shows a total budget, number of guests, party type, venue type, selectable needs such as catering, decoration, and entertainment, plus a free-text requirements field.

The screenshot displays the 'Party Budget Planner' interface within the PocketSmart application. The top navigation bar includes 'Home Planner', 'Party Planner' (active), 'Jewelry Planner', 'History', and 'Logout'. The main title 'Party Budget Planner' is centered, with the subtitle 'Plan your perfect event with AI-powered budget recommendations' below it. The form is organized into four sections: 'Basic Information' with input fields for 'Total Budget (₹)' (5000) and 'Number of Guests' (3); 'Event Details' with dropdowns for 'Party Type' (Wedding) and 'Venue Type' (Home); 'Party Needs' with toggleable options for 'Catering' (checked), 'Decoration' (unchecked), and 'Entertainment' (checked); and 'Additional Requirements' with a text area for 'Special requests, themes, dietary restrictions, etc.'. A 'Generate Budget Plan' button is at the bottom of the form. The footer shows '© 2025 PocketSmart. All rights reserved.'

Figure 8. Party Budget Planner input screen.

6.6 Party Recommendation Output

The source includes a party budget result screen and history entry showing party planning information. The documented workflow is therefore: collect event parameters → generate a budget plan → present the resulting plan.

Your Party Budget Plan

Print/Save

Budget: ₹5000.00

Venue

₹0.00

Home

Utilizing the home as the venue.

Shop on: [Google](#) [Booking](#) [Makemytrip](#) [Oyaraams](#) [Nobroker](#)

Catering

₹2000.00

Home-cooked meal

Simple, home-cooked meal for 3 people.

Shop on: [Swiggy](#) [Zomato](#)

Entertainment

₹2000.00

Streaming service subscription

One-time purchase of a movie or renting a few movies on a platform like Netflix, Amazon Prime Video, or Hotstar.

Shop on: [Amazon](#) [Flipkart](#) [Bookmyshow](#)

Board games/cards

Purchase of a new board game or card game.

Shop on: [Amazon](#) [Flipkart](#) [Bookmyshow](#)

Music playlist

Creating a celebratory playlist on Spotify or YouTube Music.

Shop on: [Amazon](#) [Flipkart](#) [Bookmyshow](#)

Small gift for couple

A small token of appreciation for the couple.

Shop on: [Amazon](#) [Flipkart](#) [Bookmyshow](#)

Contingency

₹1000.00

Unexpected expenses

Buffer for unforeseen costs.

Shop on: [Amazon](#) [Flipkart](#) [Google](#)

Total Budget

₹5000.00

Allocated

₹5000.00

Remaining

₹0.00

Venue Suggestions

Home

₹0.00

Type: Residential

Location: undefined

undefined

[Website](#) [View on Map](#)

Additional Suggestions

- Consider making the meal a potluck style if comfortable with guests to reduce catering costs.
- Look for discounts or offers on streaming services or board games.

PocketSmart AI — Project Documentation

Figure 9. Example Party Budget Plan result screen.

6.7 Jewelry Budget Planner

The Jewelry Budget Planner captures a total budget, occasion, style preferences, and an uploaded outfit image. The user can request recommendations after providing these details.

PocketSmart | Dashboard | Home Budget | Party Budget | **Jewelry Budget** | Logout

Jewelry Budget Planner

Get AI-powered jewelry recommendations within your budget for any occasion

Budget Details

Total Budget (\$)

Occasion & Preferences

Occasion

Style Preferences

Describe your style preferences, materials, colors, etc.

Upload Outfit Image



Remove Image

Get Recommendations

© 2025 PocketSmart | Smart Budget Solutions

Figure 10. Jewelry Budget Planner input screen with outfit image upload.

6.8 Jewelry Recommendation Output

The example output includes a Budget Summary, Outfit Analysis, Jewelry Recommendations, shopping links, and Styling Tips. The displayed recommendations include categories such as bracelet, ring, and watch, with descriptions, prices, and style information.

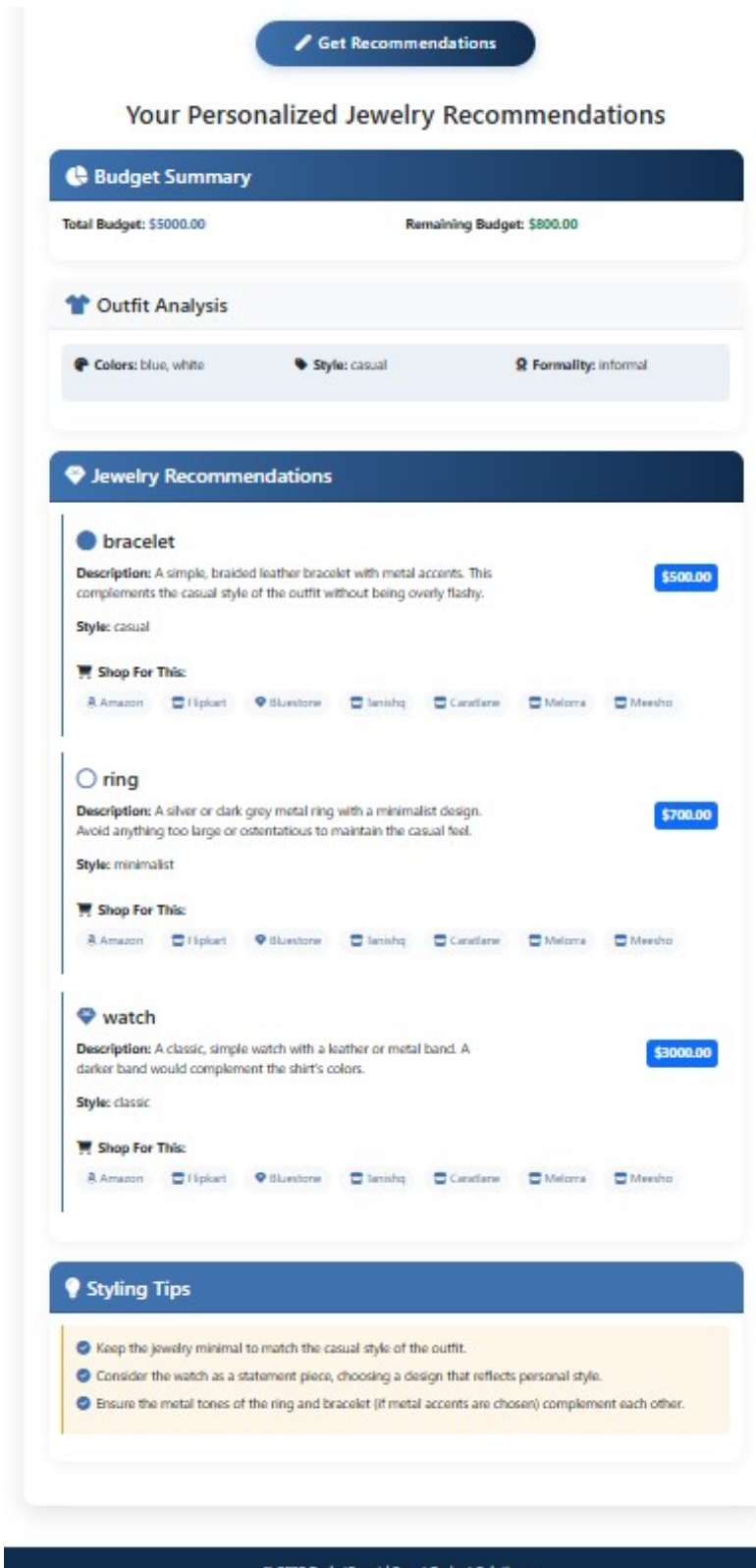


Figure 11. Example personalized jewelry recommendations.

7. Recommendation History

PocketSmart provides a dedicated Recommendation History area for viewing previous recommendations. The supplied example shows recent recommendation cards for Jewelry Budget, Party Planning, and Home Interior Budget, including dates/times and summary values such as total and remaining budget.

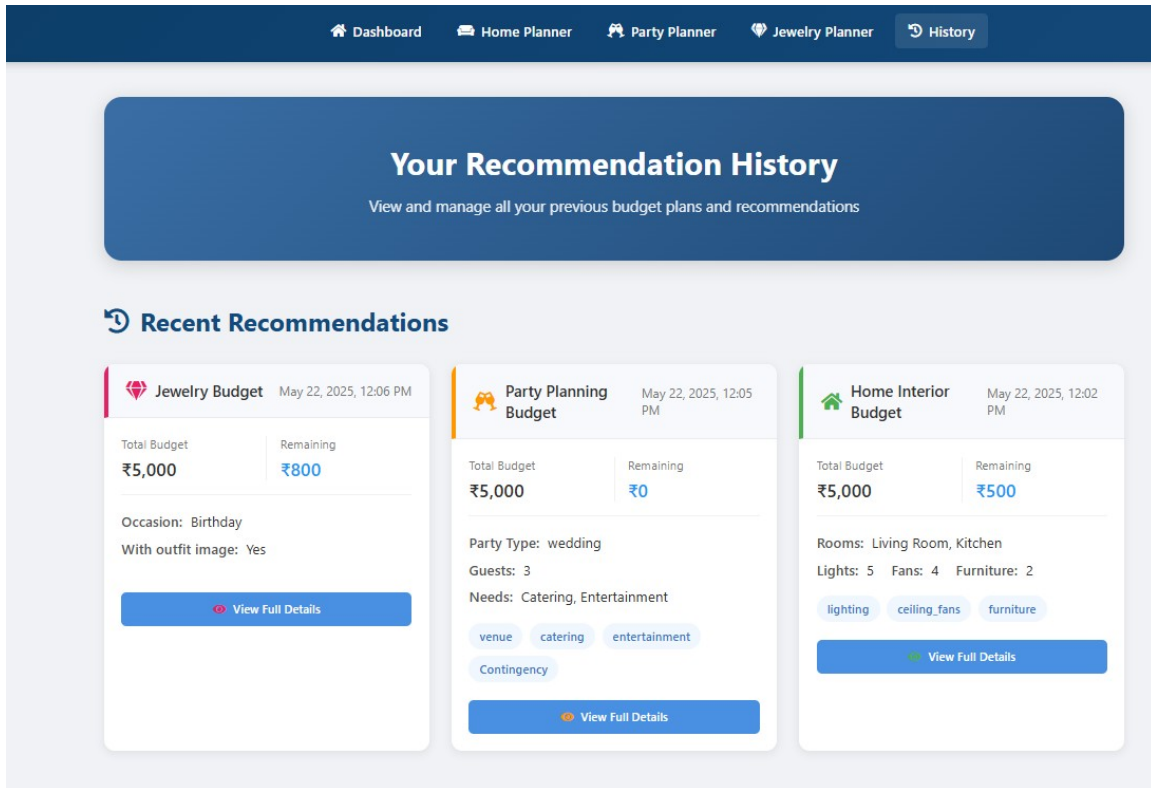


Figure 12. Recommendation History screen.

A source code excerpt also shows a FastAPI `/history` route that renders the history page using a history template and the current authenticated user.

```
# Step 7: Add a new endpoint to serve the history page
@app.get("/history", response_class=HTMLResponse)
async def history_page(request: Request, current_user: UserInDB = Depends(get_current_active_user)):
    """History page to view past recommendations"""
    return templates.TemplateResponse("history.html", {"request": request, "user": current_user})
```

Figure 13. FastAPI history endpoint shown in the source.

8. Technical Architecture

The supplied architecture diagram describes a cloud-based request flow from the user interface through AWS services to the backend, AI recommendation engine, persistence layer, user-upload storage, monitoring, and external commerce APIs.

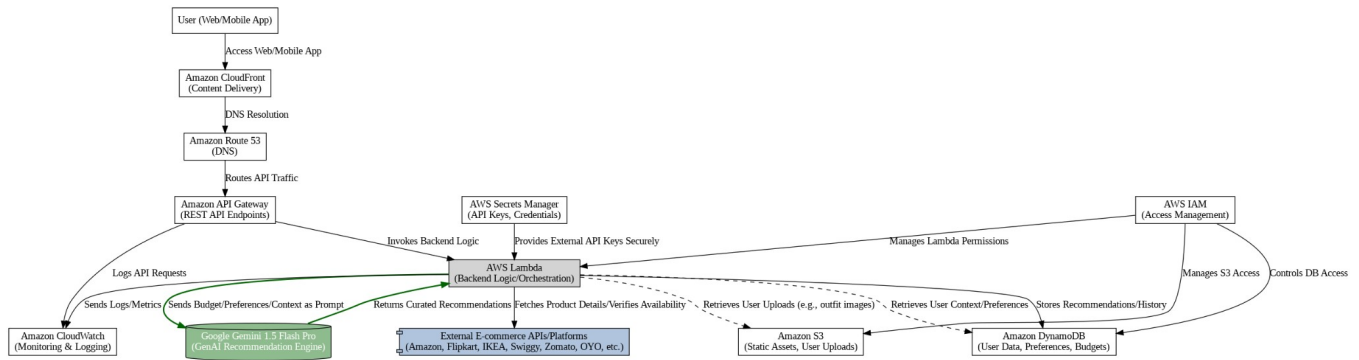


Figure 14. Cloud deployment and integration architecture shown in the source.

8.1 Architecture Components

Component	Role shown in source	Interaction
User (Web/Mobile App)	Access point for the PocketSmart experience	Accesses the application
Amazon CloudFront	Content delivery	Receives application access
Amazon Route 53	DNS resolution	Resolves/routes the application domain
Amazon API Gateway	REST API endpoints	Routes API traffic to backend logic
AWS Lambda	Backend logic / orchestration	Invokes AI, storage, database and external API operations
Google Gemini 1.5 Flash Pro	GenAI recommendation engine	Receives budget/preferences/context and returns curated recommendations
Amazon DynamoDB	User data, preferences, budgets	Stores recommendations/history and application data
Amazon S3	Static assets and user uploads	Stores/retrieves uploaded content such as outfit images
AWS Secrets Manager	API keys and credentials	Provides external API keys securely
AWS IAM	Access management	Manages Lambda permissions and access to S3/DynamoDB
Amazon CloudWatch	Monitoring and logging	Receives API logs and application metrics
External E-commerce APIs/Platforms	Product details and availability	Provides product information used in recommendations

8.2 Backend and API Layer

The source code excerpts identify FastAPI as the backend application framework. The initialization example creates a FastAPI application titled “PocketSmart: AI Budget Planner” and runs it with Uvicorn on port 8000. The source also shows HTML template responses for pages such as login and history.

```
# Main entry point
if __name__ == "__main__":
    import uvicorn
    print("Starting PocketSmart: AI Budget Planner...")
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

Figure 15. FastAPI/Uvicorn application entry point shown in the source.

```
except Exception as e:
    raise HTTPException(500, f"Error generating recommendations: {str(e)}")

def usd_to_inr(amount_usd: float, exchange_rate: float = 83.0) -> float:
    """Convert USD amount to INR using the specified exchange rate"""
    return amount_usd * exchange_rate
```

Figure 16. Error handling and USD-to-INR conversion utility shown in the source.

8.3 AI Recommendation Layer

The architecture identifies Google Gemini 1.5 Flash Pro as the generative AI recommendation engine. The documented flow indicates that budget, preferences, and context are sent as a prompt; the AI returns curated recommendations; and the backend orchestrates the surrounding storage and commerce-data operations.

9. Functional Requirements

ID	Requirement	Description
FR-01	Account access	The system shall provide sign-in and account-creation interfaces.
FR-02	Authenticated dashboard	The system shall provide an authenticated dashboard for selecting planning categories.
FR-03	Home planning input	The system shall collect budget, fixture/furniture quantities, rooms, and additional preferences.

FR-04	Party planning input	The system shall collect budget, guest count, party type, venue type, party needs, and additional requirements.
FR-05	Jewelry planning input	The system shall collect budget, occasion, style preferences, and an outfit image.
FR-06	Recommendation generation	The system shall generate a personalized budget/recommendation result from submitted planning information.
FR-07	Budget summary	The system shall display total budget and remaining budget in the recommendation result.
FR-08	Product details	The system shall display recommendation item details such as description, price and quantity where applicable.
FR-09	Shopping links	The system shall provide shopping links for displayed recommendations where available.
FR-10	Additional guidance	The system shall display additional suggestions or styling tips where applicable.
FR-11	History	The system shall provide access to previous recommendation records.
FR-12	Logout	The authenticated interface shall provide a logout action.

10. Data and Information Flow

9. User authentication establishes the current user context.
10. The selected planner collects structured inputs and free-text preferences.
11. For jewelry planning, an outfit image can also be supplied.
12. The backend receives the planner request through the API layer.
13. Lambda/backend orchestration prepares the context for the recommendation engine and related services.

14. The AI recommendation engine produces curated recommendations.
15. External commerce integrations can supply product details/availability for shopping links.
16. Recommendation/history data is stored in DynamoDB; uploaded content is handled through S3 as shown in the architecture.
17. The resulting budget plan is rendered to the user.

11. Security and Access-Control Notes

The supplied source provides visible evidence of authenticated routes, token handling, JWT configuration, AWS IAM, and AWS Secrets Manager. These are documented here as implementation elements shown in the source, rather than as a claim that the complete security posture has been independently tested.

- JWT-related configuration is shown with HS256.
- The access-token expiration value shown in the source is 30 minutes.
- The login route checks for an existing token/current user before redirecting to the dashboard.
- AWS IAM is shown as controlling Lambda permissions and access to S3/DynamoDB.
- AWS Secrets Manager is shown as the location for external API keys and credentials.

12. Error Handling and Utility Logic

A source code excerpt shows exception handling around recommendation generation, returning an HTTP 500 error with an error message when recommendation generation fails. Another utility converts a USD amount to INR using a specified exchange rate; the example default shown in the source is 83.0.

```
except Exception as e:
    raise HTTPException(500, f"Error generating recommendations: {str(e)}")

def usd_to_inr(amount_usd: float, exchange_rate: float = 83.0) -> float:
    """Convert USD amount to INR using the specified exchange rate"""
    return amount_usd * exchange_rate
```

Figure 17. Recommendation-generation error handling and currency conversion utility.

13. User Interface Summary

UI area	Description
Landing / marketing pages	Describe PocketSmart, its planners, testimonials, and a call to action to optimize a budget.
Authentication	Sign in and create-account forms.

Dashboard	Planner selection and navigation to history.
Planner forms	Sectioned input cards with budget, preferences, quantities, selections, uploads, and action buttons.
Recommendation pages	Budget summary, category cards, item details, shopping links, and guidance.
History	Recent recommendation cards with plan-specific summary information.

14. Deployment Architecture Summary

The supplied architecture depicts a managed AWS deployment in which the frontend/application access path uses CloudFront and Route 53, API requests pass through API Gateway, and Lambda performs backend orchestration. DynamoDB and S3 provide persistence/storage, while IAM, Secrets Manager, and CloudWatch provide access control, secret management, and monitoring. The recommendation engine and product information are supplied by external AI and e-commerce services.

15. Testing Considerations

The source does not contain a formal test plan. Based on the documented screens and workflows, the following test areas are directly relevant to the demonstrated functionality:

- Registration with valid and invalid account data.
- Login with valid credentials and behavior for already-authenticated users.
- Access control for authenticated planner/history routes.
- Home planner validation for budget and quantity fields.
- Party planner validation for budget, guests, event type, and selected needs.
- Jewelry planner upload and recommendation workflow.
- Recommendation generation success and error paths.
- Correct calculation/display of total and remaining budget.
- Rendering of recommendation item descriptions, prices, quantities, and links.
- Persistence and retrieval of recommendation history.
- Logout behavior.
- External service/API failure handling.

16. Limitations of This Documentation

- The supplied reference is primarily visual, so some implementation details cannot be established beyond what is visible in screenshots/code excerpts.
- A complete database schema is not explicitly documented in the source.
- A complete REST endpoint catalog and request/response schema are not provided in the source.

- Formal performance, availability, accessibility, and penetration-testing results are not provided.
- The source shows example recommendation outputs; these should not be treated as a complete catalog of all possible AI outputs.

17. Conclusion

PocketSmart is documented in the supplied project material as an AI-powered budget planning and recommendation assistant. Its demonstrated experience combines authenticated access, three scenario-specific planners, AI-generated recommendations, shopping links, recommendation history, and a cloud architecture built around FastAPI/backend orchestration, AWS managed services, external commerce integrations, and Google Gemini.

Appendix A — Source Screens Included in This Documentation

- Dashboard / planner selection
- Login and registration
- Home Interior Budget Planner
- Home recommendation output
- Party Budget Planner
- Party recommendation output
- Jewelry Budget Planner
- Jewelry recommendation output
- Recommendation History
- Cloud deployment architecture
- Selected FastAPI/JWT implementation excerpts

Appendix B — Quick Reference

Area	Quick reference
Application name	PocketSmart AI: Smart Budget & Recommendation Assistant
Primary planners	Home Interior, Party, Jewelry
Backend framework shown	FastAPI
Application server shown	Uvicorn
Authentication indication	JWT / HS256; 30-minute access-token expiration shown
AI engine shown	Google Gemini 1.5 Flash Pro
Cloud entry/services shown	CloudFront, Route 53, API Gateway, Lambda
Data/storage shown	DynamoDB and S3
Security/operations shown	IAM, Secrets Manager, CloudWatch
External integrations	E-commerce APIs/platforms such as Amazon, Flipkart, IKEA, Swiggy, Zomato, OYO, etc. as shown in the architecture